

Kafka Consumer, Vector Embeddings and sentence-transformers

Project Context

As part of the LLM Traffic Observability project, we're building a pipeline that captures LLM request/response traffic from an Envoy AI Gateway and makes it searchable using vector embeddings. This document focuses on the embeddings pipeline, semantic search, and the debugging work carried out during its implementation.

This week's goal was straightforward on paper: get captured events embedded and searchable by semantic similarity, with a synthetic dataset to prove it works before the dashboard goes on top next week. The pipeline came together cleanly, but the most useful thing I learned this week came from a bug that initially looked like a broken embedding model and turned out to be a misconfigured index. Writing it up here because I think the root cause is a good example of how an approximate-search system can *look* completely broken while every individual component is technically working correctly; plus two further issues that would only have surfaced once this runs for real instead of in a demo. Both are now fixed and tested under the conditions that would actually expose them, not just the demo conditions that were hiding them.

What got built

- A Kafka consumer that reads captured events and writes them into Postgres, batching embedding calls (32 at a time) for CPU throughput
- A pgvector schema storing prompt and completion embeddings separately, alongside metadata (`request_id`, `route`, `session_id`, `cohort`)
- A synthetic, reproducible 7-day dataset (~10k events) split across three labeled cohorts — normal traffic, drift (sessions that pivot into prompt-injection attempts), and noise (random gibberish, mainly as a control)
- A `query.py` CLI (`nearest-prompts`, `cluster-sizes`) to validate the index actually works before anyone builds a dashboard on top of it

Getting the stack running taught me to stop trusting silence

Before any of the retrieval work, just standing the pipeline up surfaced a string of small, real bugs — each one boring individually, but the pattern across them was the actual lesson. A wrong import path (`pgvector.Vector` moved to `pgvector.utils.Vector` between versions) failed loudly and was easy to fix. A worse one didn't: migrations were being resolved relative to the installed package's location, which differs between running from source and running from a `pip install`-built container. `Path.glob()` on a missing directory doesn't raise an error, it just quietly returns nothing — so migrations silently ran zero times, no exception anywhere, and the failure only surfaced two layers downstream when `register_vector` couldn't find a Postgres type that was never created. Nothing crashed until something *else* depended on the thing that silently didn't happen.

That's the pattern I'm taking forward: a clean run isn't the same as a correct run, and "no error was thrown" is not evidence that a step actually did what it was supposed to. Worth checking effects directly (row counts, `\dt`, actual index existence) rather than trusting that silence means success.

What an embedding actually is, and why it matters here

It's easy to treat `model.encode(text)` as a black box that "does AI stuff" and move on. Worth being precise about it, since the rest of the week's debugging only made sense once I stopped treating it as magic. An embedding is just a fixed-length list of ordinary floating-point numbers — 768 of them, in our case — that represents a piece of text as a single point in a 768-dimensional space. None of the individual numbers mean anything on their own; there's no "dimension 42 = sentiment." What's meaningful is the pattern across all 768 together: the embedding model is trained so that text with similar meaning ends up at nearby points, and unrelated text ends up far apart. "Similarity search" is then just geometry — measuring the angle between two points (cosine similarity) and calling closer angles "more similar."

That framing is also why we embed prompt and completion separately, which was a real design decision worth understanding rather than assuming. A request can look completely benign while the actual attack succeeds in the response — a leaked system prompt, an overly compliant answer to something that should've been refused. If we only embedded prompts, we'd be blind to exactly the cases where the interesting thing happened after the request. Embedding both sides means we can search or flag on either "what was asked" or "what actually came back," which matters for a system whose whole point is catching outcomes, not just intent.

Why the synthetic dataset needed ground-truth labels, not just volume

The cohort column (normal / drift / noise) is decided once, at data-generation time — the generator picks a category first, then writes matching text, so the label is trustworthy by construction, not inferred. That distinction turned out to matter a lot: without it, there'd be no way to tell whether search results were "roughly plausible" versus actually correct. Ten thousand anonymous rows would have let a broken search system pass a casual glance — the results don't look like an error, they look like an answer, just a wrong one. Having a known-correct label on every row turned "does this look okay" into an actual, checkable claim, which is exactly what caught the bug below.

Why Nomic instead of the originally-scoped MiniLM

The original scope named `all-MiniLM-L6-v2` (384-dim). What's actually running is `nomic-ai/nomic-embed-text-v1` (768-dim). Two concrete reasons:

- **Context window.** MiniLM truncates at 256 tokens. Completions can run well past that — a truncated embedding means anything after token 256 is invisible to search, which matters directly for a system whose job is catching things that show up *in the response*. Nomic supports up to 8192 tokens, so completions get embedded in full.
- **Retrieval performance.** Nomic outperforms MiniLM on MTEB retrieval benchmarks — the specific capability this project depends on, not just general sentence similarity.

Worth updating the spec to reflect the model actually in use, since the schema (`VECTOR( 768 )`) is now coupled to this choice.

One implementation detail worth documenting for anyone touching `embedder.py` or `query.py` later: Nomic uses **a symmetric prefixing** — text meant to be searched *for* should be prefixed `search_query:` , and text meant to be searched *within* (the corpus) should be prefixed `search_document:` . Get this backwards or inconsistent between ingestion and query time, and retrieval quality degrades, sometimes badly. This tripped me up early on, and cost some time before I traced it to a config detail rather than a code bug — the model's own `config_sentence_transformers.json` doesn't ship named prompts the way the library's `prompt_name=` argument expects, so the "correct-looking" API call was actually failing at runtime with an error that had nothing obviously to do with prefixing.

Getting the stack running taught me to stop trusting silence

Before any of the retrieval work, just standing the pipeline up surfaced a string of small, real bugs worth a mention because the pattern across them was as useful as the fixes themselves. A wrong import path (`pgvector.Vector` moved to `pgvector.utils.Vector` between versions) failed loudly and was easy to fix. A worse one didn't: migrations were being resolved relative to the installed package's location, which differs between running from source and running from a pip install-built container. `Path.glob()` on a missing directory doesn't raise an error, it just quietly returns nothing — so migrations silently ran zero times, no exception anywhere, and the failure only surfaced two layers downstream when the code couldn't find a Postgres type that was never created. Nothing crashed until something else depended on the thing that silently didn't happen. Taking that forward as a general rule: a clean run isn't the same as a correct run, and worth checking effects directly (row counts, actual index existence) rather than trusting that silence means success.

The retrieval bug: correct answers were missing entirely, not just ranked poorly

I validated retrieval with a graduated test rather than trusting a single query: exact text match first (sanity check), then a near-duplicate paraphrase, then actual injection-style queries against the drift cohort. The exact match worked perfectly (sim=1.0000, as expected). Everything past that was wrong in a strange way — querying for a prompt that was a near-verbatim match to a real drift-cohort row (one word removed) didn't surface that row at all, even at k=5. Random noise-cohort gibberish consistently outranked it.

My first assumption was the embedding prefix mismatch above. That turned out to be a real issue worth fixing, but a secondary one. The actual cause was in how the vector index was being searched, not how the vectors were computed.

What IVFFlat actually does, and why probes=1 broke things

pgvector's IVFFlat index works in two stages:

1. **At index build time**, it runs k-means clustering over the existing embeddings to produce lists centroids, and assigns every row to whichever centroid is nearest — effectively partitioning the dataset into lists buckets.
2. **At query time**, it compares your query vector against only the centroids first (cheap), picks the closest probes of them, and only then does an exact distance comparison — but only against rows in the selected buckets. Rows in unselected buckets are never compared at all.

With the default probes=1 and lists=100, every query was only ever being compared against roughly 1% of the dataset — whichever single cluster's centroid happened to be nearest to the query vector. K-means clustering has no awareness of cohort labels; it clusters purely by embedding position. So it was entirely possible — and apparently common — for the genuinely closest row to be sitting in a different bucket than the one selected, making it structurally invisible to the search, not just poorly ranked. Whatever happened to be in the one bucket that was checked "won" by default, however weak the actual match. This is meaningfully different from "the embeddings are bad" — they could have been perfect and this would still have happened, because most of the dataset was never examined in the first place.

Setting probes to check every list (SET ivfflat.probes = 10 against lists = 10) restored full recall as an immediate fix — but chasing down why the index needed that crutch in the first place surfaced a more fundamental problem with how it had been built.

The deeper issue: the index was built on an empty table, and nothing ever rebuilt it

IVFFlat's lists centroids aren't recomputed automatically — they're fixed at the moment CREATE INDEX runs, based on whatever rows happen to exist at that instant. Our schema migration creates this index as part of initial setup, before the consumer has written a single row. That means the index's clustering was computed from an **empty table** — the centroids it's built around are essentially meaningless, and nothing about the pipeline ever tells Postgres to redo that clustering once real data streams in afterward.

This didn't show up in demo testing because our workflow is make reset followed by a full bulk load — by the time anyone runs a query, all 10k rows already exist, so the degenerate index just happened to still behave reasonably. But that's an artifact of how the demo is run, not evidence the index is healthy. In an actual deployment, where the consumer runs continuously against live traffic (which is the entire stated purpose behind Week 5's dashboard), this index would keep clustering new data around centroids computed from nothing, with recall quietly getting worse over time and zero operational signal that anything was wrong — no error, no warning, nothing in the logs.

The fix: switched the index type from IVFFlat to HNSW. The relevant difference isn't just performance — it's that HNSW has no dependency on build-time data at all. Instead of computing fixed centroids once, it builds an incrementally-maintained graph of connections between vectors, so a table that starts empty and fills up over time behaves the same as one that was fully loaded before the index existed. This removes the "built on nothing" flaw structurally, rather than needing an operational workaround (a scheduled REINDEX) to paper over it.

While making that switch, I found a second, related issue worth flagging: even with HNSW, the default search-thoroughness settings are tuned for datasets much larger than ours, and some queries were coming back with a decent-but-not-actually-closest match — silently, no error, just a slightly worse answer than what actually existed in the data. Fixed this two ways: building the index with a higher-quality construction setting, and telling it to search more exhaustively at query time. Since HNSW's approximate search has some inherent randomness, I didn't trust a single passing query as proof — I re-ran the graduated test (exact match, paraphrase, injection-style) several times with different queries to confirm the fix held consistently, not just once by luck.

A second hardening pass: migrations weren't safe for more than one replica

Separately from the index issue, I found a real gap in how migrations run. Every consumer startup re-executes every *.sql migration file from scratch, relying entirely on IF NOT EXISTS guards in the SQL itself for safety. That check isn't atomic across concurrent sessions: if two consumer replicas start at the exact same moment, both can check "does this table exist," both see "no," and both attempt to create it — one succeeds, one throws a duplicate-object error and crashes. There's a second problem underneath that too: IF NOT EXISTS only protects simple create-type statements. A future migration that isn't safely re-runnable — an ALTER TABLE, a data backfill — would either error or silently double-apply on every single restart, since nothing was tracking which migrations had already run.

This wouldn't have bitten us at today's single replica, but would have turned into a startup crash loop the moment this scaled to more than one consumer instance, or the moment a migration that isn't a bare IF NOT EXISTS DDL statement got added — exactly the scenario I tested against directly below, rather than waiting to find out the hard way.

The fix, two parts: an exclusive Postgres advisory lock, acquired before any migration runs — if a second replica starts at the same moment, it doesn't race the first, it just waits until the lock is released. And a small tracking table recording which migration files have already completed successfully, checked before each one runs, so even a non-idempotent migration only ever executes once regardless of how many times the process restarts or how many replicas are running. I didn't just trust this in theory — I started two copies of the migration process at the literal same instant against the same database to check it under real contention. Neither crashed, and the tracking table ended up with exactly one row per migration file, not two.

Why ground-truth labels made all of this possible to catch

None of the retrieval investigation would have been findable without the synthetic dataset's cohort column. It's easy to eyeball search results and assume they're "roughly fine" — they don't look like an error, they look like an answer, just a wrong one. Having a known-correct label on every row turned "does this look okay" into an actual, checkable claim: does the drift-cohort row that should win, actually win. That's the real value of building the synthetic dataset with labeled cohorts before anything else — it's not just demo data, it's the reason a broken index got caught this week instead of showing up quietly during the Week 5 dashboard demo.

Where things landed

All four Week 4 acceptance criteria are passing:

- No event loss — row counts match published counts
- query.py returns plausible nearest neighbors on known prompts
- Synthetic dataset is reproducible from a single seeded command and contains all three cohorts
- Drift cohort is detectable via similarity search, confirmed after the indexing fixes above

Beyond the acceptance criteria, both hardening issues (the empty-table index and the migration race) shared the same shape: invisible under exactly the conditions we test in (fresh reset, single replica), and only real under conditions we hadn't hit yet (continuous ingestion, multiple replicas). That's exactly why I went looking for them deliberately instead of waiting for them to show up as an incident — both are fixed now, and verified against the actual failure conditions (rebuilding under live inserts, two replicas racing on startup), not just re-tested under the same demo conditions that hid them in the first place.

The main takeaway I'm carrying into next week: when something in a pipeline like this returns a wrong-looking result, or works today but seems fragile, it's worth pausing to ask what conditions the current setup is quietly relying on — and whether those conditions are actually guaranteed, or just true by coincidence of how we've been testing it.